

作为万物之灵的人，与动物的根本区别在于理性，而计算则是理性的一种重要而具体的表现形式。计算机是人类从事计算的工具，是抽象计算模型的具体物化。基于图灵模型的现代计算机，既是人类现代文明的标志与基础，更是人脑思维的拓展与延伸。

尽管计算机的性能日益提高，但这种能力在解决实际问题时能否真正得以发挥，决定性的关键因素仍在于人类自身。具体地，通过深入思考与分析获得对问题本质的透彻理解，按照长期积淀而成的框架与模式设计出合乎问题内在规律的算法，选用、改进或定制足以支撑算法高效实现的数据结构，并在真实的应用环境中充分测试、调校和改进，构成了应用计算机高效求解实际问题的典型流程与不二法门。任何一位有志于驾驭计算机的学生，都应该从这些方面入手，不断学习，反复练习，勤于总结。

本章将介绍与计算相关的基本概念，包括算法构成的基本要素、算法效率的衡量尺度、计算复杂度的分析方法与界定技巧、算法设计的基本框架与典型模式，这些也构成了全书所讨论的各类数据结构及相关算法的基础与出发点。

§ 1.1 计算机与算法

1946年问世的ENIAC开启了现代电子数字计算机的时代，计算机科学（computer science）也在随后应运而生。计算机科学的核心在于研究计算方法与过程的规律，而不仅仅是作为计算工具的计算机本身，因此E. Dijkstra及其追随者更倾向于将这门科学称作计算科学（computing science）。

实际上，人类使用不同工具从事计算的历史可以追溯到更为久远的时代，计算以及计算工具始终与我们如影相随地穿越漫长的时光岁月，不断推动人类及人类社会的进化发展。从最初颜色各异的贝壳、长短不一的刻痕、周载轮回的日影、粗细有别的绳结，以至后来的直尺、圆规和算盘，都曾经甚至依然是人类有力的计算工具。

1.1.1 古埃及人的绳索

古埃及人以其复杂而浩大的建筑工程而著称于世，在长期规划与实施此类工程的过程中，他们逐渐归纳并掌握了基本的几何度量和测绘方法。考古研究发现，公元前2000年的古埃及人已经知道如何解决如下实际工程问题：通过直线1上给定的点P，作该直线的垂线。

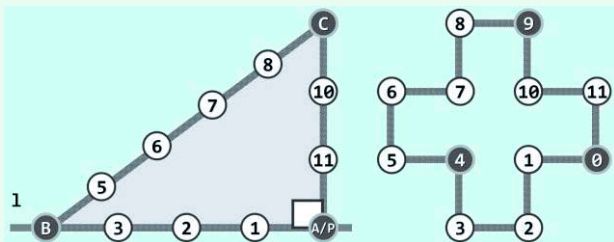


图1.1 古埃及人使用的绳索计算机及其算法

他们所采用的方法，原理及过程如图1.1所示，翻译成现代的算法语言可描述如下。

```
perpendicular(l, P)
```

输入：直线 l 及其上一点 P

输出：经过 P 且垂直于 l 的直线

1. 取12段等长绳索，依次首尾联结成环 //联结处称作“结”，按顺时针方向编号为0..11
2. 奴隶A看管0号结，将其固定于点 P 处
3. 奴隶B牵动4号结，将绳索沿直线 l 方向尽可能地拉直
4. 奴隶C牵动9号结，将绳索尽可能地拉直
5. 经过0号和9号结，绘制一条直线

算法1.1 过直线上给定点作直角

以上由古希腊人发明、由奴隶与绳索组成的这套计算工具，乍看起来与现代的电子计算机相去甚远。但就本质而言，二者之间的相似之处远多于差异，它们同样都是用于支持和实现计算过程的物理机制，亦即广义的计算机。因此就这一意义而言，将其称作“绳索计算机”毫不过分。

1.1.2 欧几里得的尺规

欧几里得几何是现代公理系统的鼻祖。从计算的角度来看，针对不同的几何问题，欧氏几何都分别给出了一套几何作图流程，也就是具体的算法。比如，经典的线段三等分过程可描述为如算法1.2所示。该算法的一个典型的执行实例如图1.2所示。

```
tripartition(AB)
```

输入：线段 AB

输出：将 AB 三等分的两个点 C 和 D

1. 从 A 发出一条与 AB 不重合的射线 ρ
2. 任取 ρ 上三点 C' 、 D' 和 B' ，使 $|AC'|| = |C'D'| = |D'B'|$
3. 联接 $B'B$
4. 过 D' 做 $B'B$ 的平行线，交 AB 于 D
5. 过 C' 做 $B'B$ 的平行线，交 AB 于 C

算法1.2 三等分给定线段

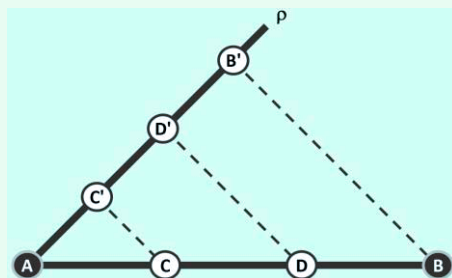


图1.2 古希腊人的尺规计算机

在以上算法中，输入为所给的直线段 AB ，输出为将其三等分的 C 和 D 点。我们知道，欧氏几何还给出了大量过程与功能更为复杂的几何作图算法，为将这些算法变成可行的实际操作序列，欧氏几何使用了两种相互配合的基本工具：不带刻度的直尺，以及半径跨度不受限制的圆规。同样地，从计算的角度来看，由直尺和圆规构成的这一物理机制也不妨可以称作“尺规计算机”。在尺规计算机中，可行的基本操作不外乎以下五类：

- 过两个点作一直线
- 确定两条直线的交点
- 以任一点为圆心，以任意半径作一个圆
- 确定任一直线和任一圆的交点（若二者的确相交）
- 确定两个圆的交点（若二者的确相交）

每一欧氏作图算法均可分解为一系列上述操作的组合，故称之为基本操作恰如其分。

1.1.3 起泡排序

D. Knuth^[3]曾指出，四分之一以上的CPU时间都用于执行同一类型的计算：按照某种约定的次序，将给定的一组元素顺序排列，比如将 n 个整数按通常的大小次序排成一个非降序列。这类操作统称排序（**sorting**）。

就广义而言，我们今天借助计算机所完成的计算任务中，有更高的比例都可归入此类。例如，从浩如烟海的万维网中找出与特定关键词最相关的前100个页面，就是此类计算的一种典型形式。排序问题在算法设计与分析中扮演着重要的角色，以下不妨首先就此做一讨论。为简化起见，这里暂且只讨论对整数的排序。

■ 局部有序与整体有序

在由一组数组成的序列 $A[0, n - 1]$ 中，满足 $A[i - 1] \leq A[i]$ 的相邻元素称作顺序的；否则是逆序的。不难看出，有序序列中每一对相邻元素都是顺序的，亦即，对任意 $1 \leq i < n$ 都有 $A[i - 1] \leq A[i]$ ；反之，所有相邻元素均顺序的序列，也必然整体有序。

■ 扫描交换

由有序序列的上述特征，我们可以通过不断改善局部的有序性实现整体的有序：从前向后依次检查每一对相邻元素，一旦发现逆序即交换二者的位置。对于长度为 n 的序列，共需做 $n - 1$ 次比较和不超过 $n - 1$ 次交换，这一过程称作一趟扫描交换。

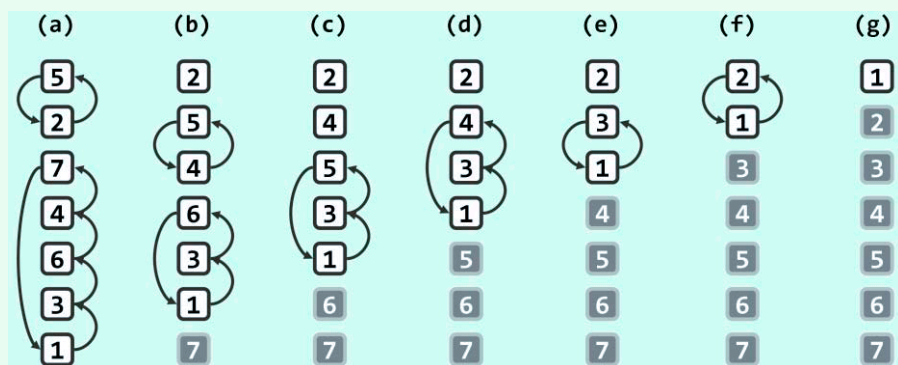


图1.3 通过6趟扫描交换对七个整数排序（其中已就位的元素以深色示意）

以图1.3(a)中由7个数组成的序列 $A[0, 6] = \{ 5, 2, 7, 4, 6, 3, 1 \}$ 为例。

在第一趟扫描交换过程中， $\{ 5, 2 \}$ 交换位置， $\{ 7, 4, 6, 3, 1 \}$ 循环交换位置，扫描交换后的结果如图(b)所示。

■ 起泡排序

可见，经过这样的一趟扫描，序列未必达到整体有序。果真如此，则可对该序列再做一趟扫描交换，比如，图(b)再经一趟扫描交换的结果如图(c)。事实上，很有可能如图(c~f)所示，需要反复进行多次扫描交换，直到如图(g)所示，在序列中不再含有任何逆序的相邻元素。多数的这类交换操作，都会使得越小（大）的元素朝上（下）方移动（习题[1-3]），直至它们抵达各自应处的位置。

排序过程中，所有元素朝各自最终位置亦步亦趋的移动过程，犹如气泡在水中的上下沉浮，起泡排序（**bubblesort**）算法也因此得名。

■ 实现

上述起泡排序的思路，可准确描述和实现为如代码1.1所示的函数bubblesort1A()。

```
1 void bubblesort1A(int A[], int n) { //起泡排序算法 (版本1A) : 0 <= n
2     bool sorted = false; //整体排序标志, 首先假定尚未排序
3     while (!sorted) { //在尚未确认已全局排序之前, 逐趟进行扫描交换
4         sorted = true; //假定已经排序
5         for (int i = 1; i < n; i++) { //自左向右逐对检查当前范围A[0, n)内的各相邻元素
6             if (A[i - 1] > A[i]) { //一旦A[i - 1]与A[i]逆序, 则
7                 swap(A[i - 1], A[i]); //交换之, 并
8                 sorted = false; //因整体排序不能保证, 需要清除排序标志
9             }
10        }
11        n--; //至此末元素必然就位, 故可以缩短待排序序列的有效长度
12    }
13 } //借助布尔型标志位sorted, 可及时提前退出, 而不致总是蛮力地做n - 1趟扫描交换
```

代码1.1 整数数组的起泡排序

1.1.4 算法

以上三例都可称作算法。那么，究竟什么是算法呢？所谓算法，是指基于特定的计算模型，旨在解决某一信息处理问题而设计的一个指令序列。比如，针对“过直线上一点作垂直线”这一问题，基于由绳索和奴隶构成的计算模型，由古埃及人设计的算法1.1；针对“三等分线段”这一问题，基于由直尺和圆规构成的计算模型，由欧几里得设计的算法1.2；以及针对“将若干元素按大小排序”这一问题，基于图灵机模型而设计的bubblesort1A()算法，等等。

一般地，本书所说的算法还应必须具备以下要素。

■ 输入与输出

待计算问题的任一实例，都需要以某种方式交给对应的算法，对所求解问题特定实例的这种描述统称为输入（input）。对于上述三个例子而言，输入分别是“某条直线及其上一点”、“某条线段”以及“由n个整数组成的某一序列”。其中，第三个实例的输入具体地由A[]与n共同描述和定义，前者为存放待排序整数的数组，后者为整数的总数。

经计算和处理之后得到的信息，即针对输入问题实例的答案，称作输出（output）。比如，对于上述三个例子而言，输出分别是“垂直线”、“三等分点”以及“有序序列”。在物理上，输出有可能存放于单独的存储空间中，也可能直接存放于原输入所占的存储空间中。比如，第三个实例即属于后一情形，经排序的整数将按非降次序存放在数组A[]中。

■ 基本操作、确定性与可行性

所谓确定性和可行性是指，算法应可描述为由若干语义明确的基本操作组成的指令序列，且每一基本操作在对应的计算模型中均可兑现。以上述算法1.1为例，整个求解过程可以明白无误地描述为一系列借助绳索可以兑现的基本操作，比如“取等长绳索”、“联结绳索”、“将绳结固定于指定点”以及“拉直绳索”等。再如算法1.2中，“从一点任意发出一条射线”、“在直线上任取三个等距点”、“联接指定两点”等，也都属于借助尺规可以兑现的基本操作。

细心的读者可能会注意到，算法1.2所涉及的操作并不都是基本的，比如，最后两句都要求“过直线外一点作其平行线”，这本身就是另一几何作图问题。幸运的是，借助基本操作的适当组合，这一子问题也可圆满解决，对应的算法则不妨称作是算法1.2的“子算法”。

从现代程序设计语言的角度，可以更加便捷而准确地理解算法的确定性与可行性。具体地，一个算法满足确定性与可行性，当且仅当它可以通过程序设计语言精确地描述，比如，起泡排序算法可以具体地描述和实现为代码1.1中的函数**bubblesort1A()**，其中“读取某一元素的内容”、“修改某一元素的内容”、“比较两个元素的大小”、“逻辑表达式求值”以及“根据逻辑判断确定分支转向”等等，都属于现代电子计算机所支持的基本操作。

■ 有穷性与正确性

不难理解，任意算法都应在执行有限次基本操作之后终止并给出输出，此即所谓算法的有穷性（**finiteness**）。进一步地，算法不仅应该迟早会终止，而且所给的输出还应该能够符合由问题本身在事先确定的条件，此即所谓算法的正确性（**correctness**）。

对以上前两个算法实例而言，在针对任一输入实例的计算过程中，每条基本操作语句仅执行一次，故其有穷性不证自明。另外，根据勾股定理以及平行等比原理，其正确性也一目了然。然而对于更为复杂的算法，这两条性质的证明往往颇需费些周折（习题[1-27]和[1-28]），有些问题甚至尚无定论（习题[1-29]）。即便是简单的起泡排序，**bubblesort1A()**算法的有穷性和正确性也不是由代码1.1自身的结构直接保证的。以下就以此为例做一分析。

■ 起泡排序

图1.3给出了**bubblesort1A()**的一次具体执行过程和排序结果，然而严格地说，这远不足以证明起泡排序就是一个名副其实的算法。比如，对于任意一组整数，经过若干趟的起泡交换之后该算法是否总能完成排序？事实上，即便是其有穷性也值得怀疑。就代码结构而言，只有在前一趟扫描交换中未做任何元素交换的情况下，外层循环才会因条件“**!sorted**”不再满足而退出。但是，这一情况对任何输入实例都总能出现吗？反过来，是否存在某一（某些）输入序列，无论做多少趟起泡交换也无济于事？这种担心并非毫无道理。细心的读者或许已注意到，在起泡交换的过程中，尽管多数时候元素会朝着各自的最终位置不断靠近，但有的时候某些元素也的确会暂时朝着远离自己应处位置的方向移动（习题[1-3]）。

证明算法有穷性和正确性的一个重要技巧，就是从适当的角度审视整个计算过程，并找出其所具有的某种不变性和单调性。其中的单调性通常是指，问题的有效规模会随着算法的推进不断递减。不变性则不仅应在算法初始状态下自然满足，而且应与最终的正确性相呼应——当问题的有效规模缩减到0时，不变性应随即等价于正确性。

那么，具体到**bubblesort1A()**算法，其单调性和不变性应如何定义和体现呢？

反观图1.3不难看出，每经过一趟扫描交换，尽管并不能保证序列立即达到整体有序，但从“待求解问题的规模”这一角度来看，整体的有序性必然有所改善。以全局最大的元素（图1.3中的整数7）为例，在第一趟扫描交换的过程中，一旦触及该元素，它必将与后续的所有元素依次交换。于是如图1.3(b)所示，经过第一趟扫描之后，该最大元素必然就位；而且在此后的各趟扫描交换中，该元素将绝不会参与任何交换。这就意味着，经过一趟扫描交换之后，我们只需要关注前面更小的那 $n - 1$ 个元素。实际上，这一结论对后续的各趟扫描交换也都成立——考查图1.3(c~g)中的元素6~2，不难验证这一点。

于是，起泡排序算法的不变性和单调性可分别概括为：**经过 k 趟扫描交换之后，最大的前 k 个元素必然就位；经过 k 趟扫描交换之后，待求解问题的有效规模将缩减至 $n - k$ 。**

反观如代码1.1所示的**bubblesort1A()**算法，外层**while**循环会不断缩减待排序序列的有效长度 n 。现在我们已经可以理解，该算法之所以能够如此处理，正是基于以上不变性和单调性。

特别地，初始状态下 $k = 0$ ，这两条性质都自然满足。另一方面，由以上单调性可知，无论如何，至多经 $n - 1$ 趟扫描交换后，问题的有效规模必将缩减至1。此时，仅含单个元素的序列，有序性不言而喻；而由该算法的不变性，其余 $n - 1$ 个元素在此前的 $n - 1$ 步迭代中业已相继就位。因此，算法不仅必然终止，而且输出序列必然整体有序，其有穷性与正确性由此得证。

■ 退化与鲁棒性

同一问题往往不限于一种算法，而同一算法也常常会有多种实现方式，因此除了以上必须具备的基本属性，在应用环境中还需从实用的角度对不同算法及其不同版本做更为细致考量和取舍。这些细致的要求尽管应纳入软件工程的范畴，但也不失为成熟算法的重要标志。

比如其中之一就是，除一般性情况外，实用的算法还应能够处理各种极端的输入实例。仍以排序问题为例，极端情况下待排序序列的长度可能不是正数（参数 $n = 0$ 甚至 $n < 0$ ），或者反过来长度达到或者超过系统支持的最大值（ $n = \text{INT_MAX}$ ），或者 $A[]$ 中的元素不见得互异甚至全体相等，以上种种都属于所谓的退化（**degeneracy**）情况。算法所谓的鲁棒性（**robustness**），就是要求能够尽可能充分地应对此类情况。请读者自行验证，对于以上退化情况，代码1.1中**bubblesort1A()**算法依然可以正确返回而不致出现异常。

■ 重用性

从实用角度评判不同算法及其不同实现方式时，可采用的另一标准是：算法的总体框架能否便捷地推广至其它场合。仍以起泡排序为例。实际上，起泡算法的正确性与所处理序列中元素的类型关系不大，无论是对于**float**、**char**或其它类型，只要元素之间可以比较大小，算法的整体框架就依然可以沿用。算法模式可推广并适用于不同类型基本元素的这种特性，即是重用性的一种典型形式。很遗憾，代码1.1所实现的**bubblesort1A()**算法尚不满足这一要求；而稍后的第2章和第3章，将使包括起泡排序在内的各种排序算法具有这一特性。

1.1.5 算法效率

■ 可计算性

相信本书的读者已经学习并掌握了至少一种高级程序设计语言，如**C**、**C++**或**Java**等。学习程序设计语言的目的，在于学会如何编写合法（即合乎特定程序语言的语法）的程序，从而保证编写的程序或者能够经过编译和链接生成执行代码，或者能够由解释器解释执行。然而从通过计算有效解决实际问题的角度来看，这只是第一个层次，仅仅做到语法正确还远远不够。很遗憾，算法所应具备的更多基本性质，合法的程序并非总是自然具备。

以前面提到的有穷性为例，完全合乎语法的程序却往往未必能够满足。相信每一位编写过程序的读者都有过这样的体验：很多合法的程序可以顺利编译链接，但在实际运行的过程中却因无穷循环或递归溢出导致异常。更糟糕的是，就大量的应用问题而言，根本就不可能设计出必然终止的算法。从这个意义讲，它们都属于不可解的问题。当然，关于此类问题的界定和研究，应归入可计算性（**computability**）理论的范畴，本书将不予过多涉及。

■ 难解性

实际上我们不仅需要确定, 算法对任何输入都能够在有穷次操作之后终止, 而且更加关注该过程所需的时间。很遗憾, 很多算法即便满足有穷性, 但在终止之前所花费的时间成本却太高。比如, 理论研究的成果显示, 大量问题的最低求解时间成本, 都远远超出目前实际系统所能提供的计算能力。同样地, 此类难解性 (intractability) 问题, 在本书中也不予过多讨论。

■ 计算效率

在“编写合法程序”这一基础之上, 本书将更多地关注于非“不可解和难解”的一般性问题, 并讨论如何高效率地解决这一层面的计算问题。为此, 首先需要确立一种尺度, 用以从时间和空间等方面度量算法的计算成本, 进而依此尺度对不同算法进行比较和评判。当然, 更重要的是研究和归纳算法设计与实现过程中的一般性规律与技巧, 以编写出效率更高、能够处理更大规模数据的程序。这两点既是本书的基本主题, 也是贯穿始终的主体脉络。

■ 数据结构

由上可知, 无论是算法的初始输入、中间结果还是最终输出, 在计算机中都可以数据的形式表示。对于数据的存储、组织、转移及变换等操作, 不同计算模型和平台环境所支持的具体形式不尽相同, 其执行效率将直接影响和决定算法的整体效率。数据结构这一学科正是以“数据”这一信息的表现形式为研究对象, 旨在建立支持高效算法的数据信息处理策略、技巧与方法。要做到根据实际应用需求自如地设计、实现和选用适当的数据结构, 必须首先对算法设计的技巧以及相应数据结构的特性了然于心, 这些也是本书的重点与难点。

§ 1.2 复杂度度量

算法的计算成本涵盖诸多方面, 为确定计算成本的度量标准, 我们不妨先从计算速度这一主要因素入手。具体地, 如何度量一个算法所需的计算时间呢?

1.2.1 时间复杂度

上述问题并不容易直接回答, 原因在于, 运行时间是由多种因素综合作用而决定的。首先, 即使是同一算法, 对于不同的输入所需的运行时间并不相同。以排序问题为例, 输入序列的规模、其中各元素的数值以及次序均不确定, 这些因素都将影响到排序算法最终的运行时间。为针对运行时间建立起一种可行、可信的评估标准, 我们不得不首先考虑其中最为关键的因素。其中, 问题实例的规模往往是决定计算成本的主要因素。一般地, 问题规模越接近, 相应的计算成本也越接近; 而随着问题规模的扩大, 计算成本通常也呈上升趋势。

如此, 本节开头所提的问题即可转化为: 随着输入规模的扩大, 算法的执行时间将如何增长? 执行时间的这一变化趋势可表示为输入规模的一个函数, 称作该算法的时间复杂度 (time complexity)。具体地, 特定算法处理规模为 n 的问题所需的时间可记作 $T(n)$ 。

细心的读者可能注意到, 根据规模并不能唯一确定具体的输入, 规模相同的输入通常都有多个, 而算法对其进行处理所需时间也不尽相同。仍以排序问题为例, 由 n 个元素组成的输入序列有 $n!$ 种, 有时所有元素都需交换, 有时却无需任何交换 (习题[1-3])。故严格说来, 以上定义的 $T(n)$ 并不明确。为此需要再做一次简化, 即从保守估计的角度出发, 在规模为 n 的所有输入中选择执行时间最长者作为 $T(n)$, 并以 $T(n)$ 度量该算法的时间复杂度。